

1. ¿Qué es un diagrama de clases? La vista estática del sistema

Si el diagrama de casos de uso nos cuenta las historias que el sistema debe protagonizar junto a sus actores, el diagrama de clases nos revela la anatomía interna de los protagonistas de esas historias. Es el modelo que captura la estructura estática del software: las clases que lo componen, los datos que almacenan, los comportamientos que exponen y las relaciones que tejen entre ellas. En esencia, mientras los casos de uso responden a la pregunta *¿qué hace el sistema?*, el diagrama de clases responde a *¿cómo está construido internamente para poder hacerlo?*.

En este primer tema sobre el modelado estructural, exploraremos el propósito del diagrama de clases dentro de UML, su naturaleza estática frente a los diagramas de comportamiento, los elementos fundamentales que aparecen en él y cómo PlantUML nos proporciona una sintaxis sencilla e intuitiva para dibujarlo y, sobre todo, para mantenerlo sincronizado con el resto de la documentación del proyecto.

1.1. El rol del diagrama de clases en UML

UML (Lenguaje Unificado de Modelado) organiza sus diagramas en dos grandes familias: los **diagramas de comportamiento** (casos de uso, secuencia, actividades, estados) y los **diagramas de estructura** (clases, objetos, componentes, despliegue, paquetes). El diagrama de clases es, sin duda, el más emblemático de los diagramas estructurales.

Su función principal es describir los tipos de objetos que existen en el sistema y las relaciones estáticas que existen entre ellos. No muestra cómo los objetos colaboran a lo largo del tiempo —eso es tarea de los diagramas de secuencia o de comunicación—, sino qué piezas componen el sistema y cómo están conectadas de manera permanente. Podemos pensarlo como el plano de ingeniería que los desarrolladores consultan para saber qué clases deben programar, qué atributos contiene cada una, qué métodos ofrecen y cómo se relacionan con otras clases.

1.2. Vista estática vs. vista dinámica

Para un ingeniero de software, es crucial distinguir la perspectiva estática de la dinámica. La vista estática —el diagrama de clases— es como una fotografía de la arquitectura del sistema: captura su esqueleto en un momento dado, independientemente de los flujos de ejecución. La vista dinámica —los diagramas de secuencia, de actividades, de estados— muestra cómo ese esqueleto cobra vida para cumplir un caso de uso concreto.

Ambas vistas se complementan. Un caso de uso como "Realizar Pedido" especifica una secuencia de interacciones entre un actor y el sistema. Cuando analizamos esa especificación textual para derivar el diagrama de clases, nos preguntamos: *¿qué objetos necesito para que esta historia sea posible?* Surgirán entonces clases como **Pedido**, **Cliente**, **Producto**, **Dirección**, **Pago**. Cada una de ellas aparecerá como un rectángulo en el diagrama de clases. Los atributos de esas clases (por ejemplo, **fechaPedido**, **total**) provienen de los datos que el caso de uso menciona o manipula. Los métodos (**calcularTotal()**, **confirmar()**) se corresponden con las acciones que el sistema debe ejecutar en los pasos del flujo.

Esta derivación desde el análisis funcional (casos de uso) hacia el modelo estructural (clases) es una de las habilidades más importantes que desarrollarán como ingenieros de software, y la abordaremos en profundidad en el tema 4 de este bloque. Por ahora, quédense con la idea de que el diagrama de clases no se inventa de la nada; debe surgir como respuesta a las necesidades funcionales documentadas en los casos de uso.

1.3. Elementos básicos de un diagrama de clases

Todo diagrama de clases se construye con un vocabulario reducido pero poderoso. Los elementos que aparecen una y otra vez son:

- **Clase:** representada como un rectángulo dividido en tres compartimentos (nombre, atributos, métodos). Es la abstracción de un concepto del dominio del problema o de la solución. Por ejemplo, `Cliente`, `Factura`, `ControladorPago`.
- **Atributo:** una propiedad o característica de la clase. Tiene un nombre y un tipo. En los diagramas puede indicarse también la visibilidad (+ público, - privado, # protegido, ~ paquete) y un valor por defecto. Ejemplo: - `nombre: String`.
- **Método:** una operación que la clase sabe realizar. También lleva visibilidad, nombre, parámetros entre paréntesis y tipo de retorno. Ejemplo: + `calcularTotal(): double`.
- **Relación:** una conexión semántica entre dos o más clases. Hay varios tipos —asociación, agregación, composición, herencia, realización, dependencia— y cada uno expresa un matiz distinto sobre la naturaleza del vínculo. Los estudiaremos en detalle en el tema 3.

1.4. Más allá del código: el diagrama de clases como artefacto de comunicación

El diagrama de clases no solo sirve para generar código o para documentar la arquitectura. Es, sobre todo, un instrumento de diálogo. Cuando un arquitecto de software propone una estructura de clases, la plasma en un diagrama para que el equipo de desarrollo la revise, la cuestione y la mejore. Cuando un desarrollador nuevo se incorpora al proyecto, el diagrama de clases le ofrece un mapa de navegación por el código fuente. Cuando se discute un cambio de requisitos, se puede evaluar su impacto observando qué clases serían afectadas.

PlantUML, con su enfoque basado en texto, convierte este artefacto en algo vivo y versionable. La guía de referencia que estamos utilizando dedica un capítulo completo (páginas 57 a 98) a los diagramas de clases, mostrando cómo declarar clases, atributos, métodos, relaciones, paquetes y todo tipo de personalizaciones visuales mediante `skinparam`. La capacidad de escribir un diagrama de clases como un archivo `.puml`, almacenarlo en Git y regenerarlo cada vez que se modifica elimina la brecha entre documentación y realidad que tanto aqueja a los proyectos de software.

1.5. Cómo PlantUML nos ayuda a modelar la estructura estática

La sintaxis de PlantUML para diagramas de clases es notablemente sencilla e intuitiva. Para declarar una clase basta con escribir su nombre, opcionalmente precedido de la palabra `class`. Si queremos añadir atributos y métodos, utilizamos los dos puntos `:` o las llaves `{ }` para agruparlos. Las relaciones se dibujan con combinaciones de guiones, puntos y flechas, cada una con su significado específico.

Un ejemplo mínimo:

???

@startuml

```
class Cliente {  
  
    • id: int  
    • nombre: String  
  
    • getNombre(): String  
    }  
    class Pedido {  
  
    • fecha: Date  
  
    • calcularTotal(): double  
    }  
    Cliente "1" -- "*" Pedido : realiza  
@enduml  
???
```

En este fragmento, definimos dos clases con sus atributos y métodos, y las vinculamos mediante una asociación con multiplicidad. La herramienta generará automáticamente el diagrama correspondiente. Como la especificación está en texto, cualquier miembro del equipo puede modificarla sin necesidad de instalar software gráfico, y los cambios quedan registrados en el sistema de control de versiones.

Durante los próximos temas, profundizaremos en cada uno de los componentes del diagrama de clases: la riqueza de las relaciones, la semántica de la agregación y la composición, el papel de las interfaces y las clases abstractas, y, muy especialmente, el proceso de derivación desde los casos de uso. Al final de este bloque, serán capaces de leer y escribir diagramas de clases con soltura, y habrán incorporado a su caja de herramientas una técnica esencial para el diseño de software robusto y bien comunicado.

2. Clases, atributos y métodos: los bloques de construcción del modelo estructural

Si el diagrama de clases es el plano del sistema, las clases son sus ladrillos, y los atributos y métodos, las vetas y la argamasa que les dan consistencia y propósito. Cada clase encapsula un concepto del dominio —una entidad, un rol, un proceso— y lo dota de estado (atributos) y comportamiento (métodos). Dominar su notación y su semántica es el primer paso para construir modelos estructurales que sean fieles a la realidad del negocio y útiles para quienes deben implementarlos.

En este tema desgranaremos cada uno de estos elementos, revisaremos cómo la guía de PlantUML nos permite declararlos con precisión y flexibilidad, y exploraremos las variantes que UML ofrece para enriquecer la expresión del diseño: visibilidad, miembros estáticos, miembros abstractos y organización avanzada del cuerpo de la clase.

2.1. La clase: mucho más que un rectángulo con nombre

En UML, una clase se representa como un rectángulo dividido en tres compartimentos. El superior contiene el nombre de la clase (y opcionalmente estereotipos como `<<entity>>`, `<<control>>` o `<<boundary>>`). El compartimento central aloja los atributos, y el inferior, los métodos. Esta división tripartita es el estándar visual, pero PlantUML es capaz de generar automáticamente los compartimentos a partir de una declaración textual.

La declaración más simple de una clase es simplemente escribir su nombre:

```
???  
@startuml  
class Cliente  
@enduml  
???
```

Esto dibuja un rectángulo con el nombre "Cliente" y los compartimentos de atributos y métodos vacíos. Sin embargo, rara vez queremos una clase sin contenido. La guía de PlantUML (páginas 60-63) detalla varias maneras de añadir miembros a una clase: usando dos puntos para declaraciones sueltas, o bien llaves `{ }` para agrupar atributos y métodos de forma más ordenada.

```
???  
@startuml  
class Cliente {  
  
    • id: int  
    • nombre: String  
  
    • getNombre(): String  
    • setNombre(n: String): void  
}  
@enduml  
???
```

2.2. Atributos: el estado que perdura

Un atributo es una propiedad que caracteriza a los objetos de una clase. Toda instancia de `Cliente` tendrá un valor para `id` y otro para `nombre`. Los atributos representan el estado del sistema en un momento dado, y suelen corresponderse con columnas de una base de datos, campos de un formulario o datos que fluyen entre componentes.

En PlantUML, los atributos se declaran dentro de las llaves de la clase o bien con la notación `NombreClase : atributo`. La sintaxis completa incluye, por este orden:

1. **Visibilidad:** `+` (público), `-` (privado), `#` (protegido), `~` (paquete).
2. **Nombre:** un identificador significativo.
3. **Tipo:** separado por dos puntos (opcional pero recomendable).
4. **Multiplicidad:** entre corchetes si se trata de una colección.
5. **Valor por defecto:** tras un signo igual (opcional).
6. **Propiedades adicionales:** entre llaves, como `{ordered}`, `{unique}`, `{readOnly}`.

Ejemplo de un atributo completo:

???

@startuml

```
class Producto {  
    • codigo: String [1] {readOnly}  
    • precio: double = 0.0  
    • etiquetas: String [*] {ordered}  
}  
@enduml  
???
```

Aquí **codigo** es privado, de tipo **String**, obligatorio y de solo lectura. **precio** es un **double** con valor por defecto 0.0. **etiquetas** es una lista ordenada de cadenas. La guía de PlantUML permite omitir cualquier parte que no sea relevante en el nivel de abstracción en que estemos modelando; en fases tempranas de diseño, podemos mostrar solo los nombres de los atributos sin preocuparnos aún por los tipos.

2.3. Métodos: el comportamiento que transforma

Los métodos especifican lo que una clase sabe *hacer*. Pueden verse como las operaciones que el sistema ejecuta cuando un actor dispara un caso de uso, pero a diferencia de los pasos de un flujo de interacción, aquí se definen de manera abstracta, sin detallar el algoritmo interno. Solo importa su firma: nombre, parámetros, tipo de retorno y visibilidad.

La notación en PlantUML es similar a la de los atributos, pero incluye paréntesis después del nombre del método (incluso si no hay parámetros) y, opcionalmente, el tipo de retorno tras dos puntos.

???

@startuml

```
class CalculadoraImpuestos {  
    • calcular(importe: double): double  
    • validarCodigoPostal(cp: String): boolean  
}  
@enduml  
???
```

Observen cómo **calcular** es público y devuelve un **double**, mientras que **validarCodigoPostal** es privado (quizás solo se usa internamente) y devuelve un booleano. La distinción entre métodos públicos y privados es esencial para encapsular la lógica interna y exponer solo lo que otras clases necesitan conocer.

2.4. Visibilidad: controlando el acceso a los miembros

UML define cuatro niveles de visibilidad para atributos y métodos, y PlantUML los representa con los siguientes caracteres:

Símbolo	Visibilidad	Icono (si se activa)	Significado
---------	-------------	----------------------	-------------

Símbolo	Visibilidad	Icono (si se activa)	Significado
+	Público	Círculo verde	Accesible desde cualquier clase
-	Privado	Cuadrado rojo	Accesible solo dentro de la propia clase
#	Protegido	Diamante azul	Accesible desde la clase y sus subclases
~	Paquete	Triángulo amarillo	Accesible desde cualquier clase del mismo paquete

La guía de PlantUML (páginas 61-62) explica que, por defecto, los iconos de visibilidad se dibujan junto a cada atributo y método. Si el diagrama resulta demasiado recargado, podemos ocultarlos con `skinparam classAttributeIconSize 0`. En mis diagramas de alto nivel, suelo ocultarlos para mantener la limpieza; en diagramas detallados, los muestro porque son información valiosa para el desarrollador.

2.5. Miembros estáticos y abstractos

Dentro de una clase pueden existir miembros que pertenecen a la clase en sí misma y no a sus instancias (estáticos), o miembros que carecen de implementación y deben ser definidos por las subclases (abstractos). PlantUML nos permite marcarlos con los modificadores `{static}` y `{abstract}` (páginas 62-63). El modificador `{classifier}` es sinónimo de `{static}`.

```

???
@startuml
class Contador {
{static} -total: int
{abstract} +incrementar(): void
}
@enduml
???
```

En este ejemplo, `total` es un atributo de clase (compartido por todas las instancias) y `incrementar()` es un método abstracto que deberán implementar las subclases. La notación UML tradicional mostraría los miembros abstractos en cursiva y los estáticos subrayados, pero PlantUML utiliza las etiquetas entre llaves para que no haya ambigüedad.

2.6. Organización avanzada del cuerpo de la clase

A medida que una clase acumula atributos y métodos, conviene agruparlos para facilitar la lectura. PlantUML permite insertar separadores dentro de la definición de la clase usando líneas con `--`, `..`, `==` o `___` (páginas 62-64). Cada separador puede ir seguido de un título que describa el grupo.

```

???
@startuml
class Usuario {
.. Datos personales ..
    • nombre: String
    • email: String
___ Seguridad ___
}
```

- passwordHash: String
 - ultimoAcceso: Date
- == Métodos públicos ==
- registrarse(): void
 - autenticar(credenciales: Credenciales): boolean
- }
- @enduml
- ???

Esta técnica es especialmente útil para clases complejas, como las entidades de un modelo de dominio, donde hay decenas de atributos y métodos. Los separadores guían al lector hacia la información relevante sin tener que escrutar una lista interminable.

2.7. De los casos de uso a las clases, atributos y métodos

Aunque dedicaremos un tema completo a la derivación desde los casos de uso, conviene anticipar la conexión: cada sustantivo relevante que aparece en la especificación textual de un caso de uso es un candidato a convertirse en clase. Por ejemplo, en el flujo básico de "Realizar Pedido" encontramos "Cliente", "Producto", "Pedido", "Dirección". Estos sustantivos se convierten en las clases de nuestro modelo estructural.

Los datos que se mencionan en el flujo —"nombre del cliente", "precio del producto", "fecha del pedido"— se transforman en atributos. Las acciones que el sistema realiza —"calcular total", "validar stock", "confirmar pago"— dan lugar a métodos.

Esta conexión asegura que el modelo de clases no sea una invención arbitraria, sino un reflejo fiel de lo que el sistema necesita hacer. Y puesto que tanto los casos de uso como el diagrama de clases se escriben en PlantUML, ambos artefactos pueden versionarse conjuntamente y mantenerse sincronizados sin fricción.

Con este dominio de las clases, sus atributos y métodos, estamos listos para abordar el tejido que une unas clases con otras: las relaciones estructurales, que son el tema al que dedicaremos la siguiente sección.

3. Relaciones entre clases: el tejido estructural del sistema

Si las clases son los nodos de nuestra red conceptual, las relaciones son los hilos que las unen y les dan sentido colectivo. Una clase aislada no basta; lo que convierte un conjunto de clases en un modelo de software es precisamente la manera en que se vinculan entre sí. UML define varios tipos de relaciones, cada una con una semántica precisa que va mucho más allá de "hay una línea entre A y B". Como ingenieros, debemos elegir la relación correcta para cada vínculo del dominio, porque esa decisión condiciona cómo se generará el código, cómo se gestionará la persistencia y cómo evolucionará el sistema ante cambios futuros.

En este tema recorreremos todos los tipos de relación que PlantUML nos permite dibujar en un diagrama de clases: asociación simple, agregación, composición, herencia, realización de interfaces y dependencia. Veremos su notación, su significado estructural, cuándo aplicar cada una y cómo expresarlas con la sintaxis textual de PlantUML, apoyándonos en los ejemplos y referencias de la guía (páginas 58 a 66, principalmente). Al final, tendrán un mapa completo de las conexiones entre clases que les permitirá modelar la arquitectura estática de cualquier sistema con rigor y claridad.

3.1. La asociación simple: el vínculo semántico básico

Una asociación es una conexión estructural entre dos clases que indica que los objetos de una clase están relacionados con los objetos de la otra de forma significativa para el dominio. Es la relación más genérica y, por tanto, la más frecuente. En un diagrama de clases, se representa con una línea continua entre las dos clases.

Por ejemplo, si decimos que un **Cliente** realiza **Pedidos**, estamos afirmando que existe una asociación entre **Cliente** y **Pedido**. Esa afirmación implica que en tiempo de ejecución habrá objetos de ambas clases vinculados: cada pedido pertenece a un cliente, y un cliente puede tener múltiples pedidos.

La notación PlantUML para la asociación simple emplea dos guiones `--`. Podemos añadir un nombre a la asociación, especificar la dirección de lectura con un triángulo `>` o `<`, e indicar la multiplicidad en cada extremo.

```
???  
@startuml  
class Cliente  
class Pedido  
Cliente "1" -- "*" Pedido : realiza  
@enduml  
???
```

En este ejemplo, la multiplicidad **1** junto a **Cliente** indica que un pedido está asociado exactamente a un cliente. La multiplicidad ***** junto a **Pedido** significa que un cliente puede estar asociado a cero o más pedidos. La etiqueta **realiza** es opcional y aclara la naturaleza de la asociación. La guía de PlantUML (página 59) muestra que también podemos invertir la dirección de lectura añadiendo `>` o `<` a la etiqueta, lo que ayuda a entender quién actúa sobre quién.

La asociación no implica dependencia de existencia: un pedido puede existir sin un cliente asociado (si la multiplicidad fuese **0..1**), y un cliente puede existir sin pedidos. Esta independencia es lo que diferencia la asociación simple de la agregación y la composición, como veremos a continuación.

3.2. Agregación y composición: cuando la parte depende del todo

En muchos dominios existen relaciones "todo-parte": un pedido se compone de líneas de pedido, un departamento agrupa empleados, un vehículo consta de motor y ruedas. UML captura estos vínculos mediante dos variantes de asociación reforzada: la agregación y la composición.

3.2.1. Agregación (rombo blanco)

La agregación es una relación todo-parte en la que la parte **puede existir independientemente** del todo. Se representa con una línea que lleva un rombo vacío en el extremo del todo. En PlantUML, el símbolo es `o--`.

???

@startuml

class Departamento

class Empleado

Departamento "1" o-- "*" Empleado : pertenece a

@enduml

???

Aquí, un **Empleado** puede pertenecer a un **Departamento**, pero si el departamento se disuelve, el empleado no desaparece; simplemente se queda sin departamento o se reasigna a otro. La metáfora es: el todo "agrega" las partes, pero éstas conservan su identidad y ciclo de vida propios.

Otro ejemplo típico: un **Equipo** de proyecto agrega **Ingenieros**. Si el proyecto termina, los ingenieros siguen existiendo y pueden incorporarse a otros equipos.

3.2.2. Composición (rombo negro)

La composición es una relación todo-parte más fuerte que la agregación. En ella, la parte **no puede existir sin el todo**. Se representa con un rombo relleno en el extremo del todo. En PlantUML, el símbolo es `*--`.

???

@startuml

class Pedido

class LineaPedido

Pedido "1" -- "*" LineaPedido : se compone de

@enduml

???

En este caso, las líneas de pedido no tienen sentido fuera del pedido al que pertenecen. Si se elimina un pedido, sus líneas asociadas también deben desaparecer. La composición indica que la responsabilidad del ciclo de vida de las partes recae sobre el todo: el objeto compuesto crea, gestiona y destruye sus componentes.

La guía de PlantUML (página 58) muestra estos símbolos y aconseja usarlos con propiedad. Un error frecuente es emplear composición por defecto para cualquier relación todo-parte, cuando muchas de esas relaciones son en realidad agregaciones. La pregunta clave es: *¿la parte sobrevive si el todo desaparece?* Si la respuesta es sí, usen agregación. Si es no, usen composición.

3.3. Herencia: la relación "es-un"

La herencia —también llamada generalización— es el mecanismo que permite a una clase (la subclase o clase hija) reutilizar la estructura y el comportamiento de otra (la superclase o clase padre), especializándola o extendiéndola. La subclase hereda todos los atributos y métodos del padre, y puede añadir nuevos o redefinir los existentes.

En UML, la herencia se representa con una línea continua y una punta de flecha hueca (triángulo vacío) que apunta desde la subclase hacia la superclase. En PlantUML, el símbolo es `<|--`.

???

@startuml

class Vehiculo {

- matricula: String
 - acelerar(): void
- }
- class Coche extends Vehiculo {
- numPuertas: int
 - abrirMaletero(): void
- }
- class Moto extends Vehiculo {
- tieneSidecar: boolean
- }
- @enduml
- ???

Observen que **Coche** y **Moto** heredan **matricula** y **acelerar()** de **Vehiculo**, pero cada una añade sus propias particularidades. La herencia permite tratar objetos de las subclases como si fueran del tipo padre (polimorfismo), lo que es uno de los pilares del diseño orientado a objetos.

La guía de PlantUML (páginas 58-59) también permite usar la palabra clave **extends** en lugar del símbolo `<|--`, lo que puede resultar más legible para quienes prefieren una sintaxis más cercana al código:

???

@startuml

class Vehiculo

class Coche extends Vehiculo

class Moto extends Vehiculo

@enduml

???

Ambas notaciones producen el mismo diagrama.

3.4. Realización: implementando una interfaz

La realización es la relación que existe entre una clase y la interfaz que implementa. Una interfaz define un conjunto de métodos sin implementar (un contrato), y las clases que la realizan se comprometen a proporcionar una implementación concreta para todos ellos.

En PlantUML, la interfaz se puede declarar con la palabra **interface** o con el símbolo de círculo (notación "lollipop"). La relación de realización se dibuja con una línea discontinua y una punta de flecha hueca:

`<|..`

???

@startuml

```
interface IPagable {  
    • calcularImporte(): double  
    • procesarPago(): boolean  
}  
class Factura implements IPagable  
class Recibo implements IPagable  
@enduml  
???
```

Aquí, **Factura** y **Recibo** realizan la interfaz **IPagable**. Cualquier cliente que trabaje con **IPagable** podrá tratar indistintamente con facturas y recibos, sin preocuparse de la implementación concreta. Esto es especialmente útil para desacoplar módulos y aplicar el principio de inversión de dependencias.

La guía de PlantUML (páginas 67 y 76) muestra también la notación compacta de interfaz como un círculo unido a la clase que la implementa, alternativa que ahorra espacio en el diagrama.

3.5. Dependencia: un uso puntual y débil

La dependencia es la relación más efímera y sutil de UML. Se produce cuando una clase A usa a una clase B de manera temporal o indirecta: por ejemplo, como parámetro de un método, como variable local o como tipo de retorno. A diferencia de la asociación, la dependencia no implica un vínculo estructural permanente entre los objetos de las clases involucradas.

En PlantUML, la dependencia se dibuja con una línea discontinua y una flecha abierta: **..>**.

???

@startuml

```
class ControladorPedido {  
    • confirmar(p: Pedido): void  
}  
class ServicioEmail {  
    • enviarConfirmacion(destinatario: String): void  
}  
ControladorPedido ..> ServicioEmail : usa  
@enduml  
???
```

En este ejemplo, **ControladorPedido** usa **ServicioEmail** probablemente dentro del método **confirmar**, pero no mantiene una referencia permanente a él; puede crearlo, llamarlo y descartarlo. La dependencia se satisface a nivel de método, no a nivel de instancia. La guía de PlantUML (página 58) recoge esta notación y la distingue claramente de la asociación.

3.6. Multiplicidad y navegabilidad: afinando las conexiones

Tanto las asociaciones como las agregaciones y composiciones pueden —y deben— precisarse con multiplicidad en cada extremo. La multiplicidad indica cuántas instancias de una clase se relacionan con cuántas de la otra. Los valores posibles incluyen números concretos (1, 2), rangos (0..1, 3..5), el asterisco (* significa cero o muchos) y combinaciones (1..*).

PlantUML coloca la multiplicidad entre comillas en el extremo correspondiente de la asociación, como ya vimos en los ejemplos anteriores.

La navegabilidad, por su parte, indica en qué dirección es posible recorrer la asociación. Una flecha en el extremo de la línea señala que desde esa clase se puede acceder a la otra, pero no necesariamente al revés. Si no hay puntas de flecha, la asociación es navegable en ambos sentidos. En PlantUML, añadir > o < en la línea determina la navegabilidad.

```
???\n@startuml\nclass Cliente\nclass HistorialPedidos\nCliente "1" --> "1" HistorialPedidos : consulta\n@enduml\n???
```

La flecha a la derecha de la línea indica que **Cliente** puede navegar hacia **HistorialPedidos** (es decir, desde un cliente puedo llegar a su historial), pero no al revés a menos que se explicita.

3.7. Cómo elegir la relación correcta: una guía práctica

Frente a un caso de uso concreto, la elección de la relación adecuada surge de preguntarse:

- ¿A necesita una referencia permanente a B? → **Asociación**.
- ¿A está compuesto por B y B no puede vivir sin A? → **Composición**.
- ¿A está compuesto por B pero B puede existir por sí mismo? → **Agregación**.
- ¿A es un tipo más específico de B? → **Herencia**.
- ¿A se compromete a cumplir el contrato definido por B? → **Realización** (interfaz).
- ¿A usa a B solo de pasada, sin conservarla? → **Dependencia**.

No hay una respuesta única, pero sí hay malas decisiones que generan acoplamientos indeseados o modelos que no reflejan la realidad del negocio. Debatir estas relaciones en equipo, con el diagrama de clases en la pizarra o en el editor de PlantUML, es una de las actividades más productivas que pueden tener como ingenieros de software.

3.8. Versatilidad de la notación PlantUML para todas las relaciones

La guía de referencia (páginas 58-60 y secciones posteriores) documenta la sintaxis completa para cada tipo de relación. Además, PlantUML permite personalizar el estilo de las líneas con colores, grosores y patrones (punteado, discontinuo, etc.) usando las notaciones con corchetes [#color,thickness=n] o los estereotipos de línea [bold], [dashed], [dotted], [hidden] y [plain]. Esto puede ser muy útil para diferenciar visualmente la criticidad de ciertas asociaciones o para diagramas orientados a la presentación ejecutiva.

Con este conocimiento de las relaciones estructurales, estamos en condiciones de abordar el verdadero desafío: cómo extraer este modelo de clases a partir de los casos de uso. Ese será precisamente el tema central de la siguiente sección.

4. Del caso de uso al diagrama de clases: identificando clases a partir del análisis funcional

Hemos consolidado ya los dos pilares del modelado UML: sabemos capturar las funcionalidades del sistema mediante casos de uso y conocemos la sintaxis y la semántica de los diagramas de clases. Ahora ha llegado el momento de tender el puente entre ambas vistas. Este es, desde mi experiencia como director de proyecto, el proceso intelectual más determinante en la fase de análisis y diseño: **derivar el modelo estructural del sistema a partir de las necesidades funcionales expresadas en los casos de uso**.

Si se realiza correctamente, el diagrama de clases resultante no será un invento arbitrario del arquitecto de software, sino un reflejo fiel de lo que los actores esperan que el sistema haga. Cada clase tendrá una razón de ser vinculada a una historia de usuario; cada atributo estará justificado por un dato que fluye en algún caso de uso; cada método responderá a una acción que el sistema debe ejecutar; cada relación entre clases será la materialización de una colaboración necesaria entre objetos para cumplir un objetivo del actor.

En este tema, recorreremos un método sistemático para extraer clases, atributos, métodos y relaciones a partir de la especificación textual de los casos de uso. Utilizaremos como base la especificación de "Realizar Pedido" que ya conocemos y mostraremos, paso a paso, cómo se transforma en un modelo de clases expresado en PlantUML.

4.1. La esencia del método: escuchar al caso de uso

La especificación textual de un caso de uso describe, en lenguaje natural, una secuencia de interacciones entre actores y sistema. Si leemos con atención, encontraremos tres categorías lingüísticas que nos orientan en la identificación de elementos del modelo estructural:

- **Sustantivos:** suelen corresponder a clases o atributos.
- **Verbos:** suelen corresponder a métodos o, si son acciones que el sistema realiza como un todo, a responsabilidades de una clase.
- **Frases posesivas o de pertenencia** ("el pedido del cliente", "las líneas del pedido"): sugieren relaciones estructurales entre clases, a menudo asociaciones, agregaciones o composiciones.

La técnica no es mágica ni completamente automática, pero aplicada con criterio y discusión en equipo produce modelos notablemente alineados con el dominio del problema.

Analicemos un fragmento del flujo básico de "Realizar Pedido" que ya usamos en el bloque de casos de uso:

1. El Cliente solicita iniciar un nuevo pedido.
2. El Sistema muestra el catálogo de productos disponibles.
3. El Cliente selecciona uno o varios productos y las cantidades deseadas.

4. El Sistema agrega los productos al carrito de compra y muestra un resumen parcial.
- ...
5. El Sistema procesa el pago, actualiza el inventario y envía un correo de confirmación.

A lo largo de este tema, este fragmento nos servirá de ejemplo conductor.

4.2. Identificación de clases candidatas a partir de los sustantivos

El primer paso consiste en extraer todos los sustantivos y sintagmas nominales que aparecen en los flujos del caso de uso (básico, alternativos y de excepción). No todos se convertirán en clases; algunos serán atributos de otras clases, otros serán actores externos y otros serán conceptos irrelevantes para el sistema. Pero una lista inicial exhaustiva nos da la materia prima para la discusión.

Del fragmento anterior obtenemos:

Sustantivo	Posible significado en el modelo
Cliente	Actor (externo al sistema, no lo modelamos como clase, o bien lo modelamos como clase si guardamos datos del cliente)
Pedido	Candidato a clase (entidad central del caso de uso)
Sistema	Nosotros mismos (el sistema bajo diseño, no es clase)
Catálogo	Candidato a clase (o quizás una interfaz o controlador que gestiona productos)
Producto	Candidato a clase (entidad que representa un artículo del catálogo)
Cantidad	Atributo (cantidad de un producto en una línea de pedido)
Carrito	Candidato a clase (contenedor temporal de productos antes de confirmar el pedido)
Resumen	Concepto efímero (posiblemente un DTO o simplemente una vista, no una clase persistente)
Pago	Candidato a clase (entidad que representa la transacción económica)
Inventario	Candidato a clase (o un subsistema que gestiona el stock)
Correo	Atributo o clase (dependiendo de la complejidad; puede ser una clase CorreoElectronico que se asocia a un servicio de envío)

Tras esta primera criba, los candidatos a clase que suelen consolidarse son: **Pedido**, **Producto**, **Carrito**, **Pago**, **Cliente** (si se almacenan sus datos), **LineaPedido** (surge al refinar la relación entre Pedido y Producto con cantidad), y posiblemente **Inventario** y **ServicioCorreo**. No todos pasarán el filtro final, pero vamos a analizar cada uno con rigor.

4.3. Identificación de atributos

Una vez tenemos las clases candidatas, las enriquecemos buscando en el caso de uso los datos que se mencionan explícitamente. En el flujo de "Realizar Pedido" aparecen: "cantidad", "dirección de envío", "precio", "total", "fecha del pedido", "email del cliente", "estado del pedido". Estos datos se asignan a las clases correspondientes.

Por ejemplo, `Pedido` tendrá atributos como:

- `fechaPedido: Date`
- `total: double`
- `estado: String` (pendiente, confirmado, enviado...)
- `direccionEnvio: String`

`Producto` tendrá:

- `codigo: String`
- `nombre: String`
- `precio: double`

`LineaPedido` tendrá:

- `cantidad: int`
- `precioUnitario: double` (el precio en el momento de la compra)

`Cliente` (si se modela) tendrá:

- `id: int`
- `nombre: String`
- `email: String`

Una práctica sana es revisar también las postcondiciones, porque suelen mencionar el estado final persistente. Si la postcondición dice "el inventario se ha actualizado", eso sugiere que `Producto` quizás deba llevar un atributo `stock: int`.

4.4. Identificación de métodos

Los verbos de acción del caso de uso —especialmente aquellos que describen lo que el sistema *hace* en respuesta a una acción del actor— se convierten en métodos de las clases adecuadas. Asignar un método a la clase correcta es una decisión de diseño que debe basarse en el principio de **experto en información**: el método debe residir en la clase que posee los datos necesarios para llevarlo a cabo.

Analicemos algunas acciones del flujo y su asignación:

- "El Sistema muestra el catálogo de productos disponibles" → `Catalogo.mostrarProductosDisponibles(): List<Producto>` (o `Producto.buscarDisponibles(): List<Producto>`).
- "El Sistema agrega los productos al carrito" → `Carrito.agregarProducto(p: Producto, cantidad: int): void`.
- "El Sistema calcula el costo de envío e impuestos y muestra el total" → `Pedido.calcularTotal(): double` (método que internamente delegará en otros objetos).
- "El Sistema procesa el pago" → `Pago.procesar(): boolean` (o `ServicioPago.realizarTransacción(p: Pedido): boolean`).
- "El Sistema actualiza el inventario" → `Producto.decrementarStock(cantidad: int): void`.
- "El Sistema envía un correo de confirmación" → `ServicioCorreo.enviarConfirmacion(destinatario: String, pedido: Pedido): void`.

Observen cómo cada acción se traduce en un método responsable, y cómo surgen nuevas clases de soporte (como **ServicioPago** o **ServicioCorreo**) que no son entidades del dominio puro, sino servicios técnicos necesarios para completar el caso de uso. Esto es perfectamente válido en un diagrama de clases de diseño.

4.5. Identificación de relaciones

Con las clases, atributos y métodos sobre la mesa, el siguiente paso es conectarlas mediante las relaciones adecuadas. Recurrimos de nuevo al caso de uso: las frases que indican posesión, pertenencia o colaboración nos guían.

- "El pedido del cliente" → asociación entre **Cliente** y **Pedido** con multiplicidad **1** a **0..***.
- "El pedido contiene líneas de pedido" → composición entre **Pedido** y **LineaPedido**, porque una línea de pedido no puede existir sin un pedido que la contenga.
- "Cada línea de pedido referencia un producto" → asociación entre **LineaPedido** y **Producto**.
- "El pago está asociado a un pedido" → asociación uno a uno entre **Pedido** y **Pago**.
- "El carrito contiene productos" → agregación o asociación entre **Carrito** y **Producto** (el producto existe independientemente del carrito, así que no hay composición).
- "El sistema envía un correo" → dependencia de **Pedido** o **Cliente** hacia **ServicioCorreo**.

También podemos identificar herencias o interfaces. Si en el caso de uso aparecen múltiples formas de pago (tarjeta, PayPal, transferencia), podemos modelar una interfaz **MetodoPago** realizada por clases concretas **PagoTarjeta**, **PagoPayPal**, etc. El caso de uso nos da la pista de que el comportamiento varía en un punto de extensión.

4.6. Representación en PlantUML del modelo derivado

Voy a plasmar ahora, en un solo diagrama de clases de PlantUML, el resultado del análisis anterior. Incluiré las clases, atributos, métodos y relaciones que hemos identificado, usando la notación y símbolos que ya dominamos.

???

@startuml

class Cliente {

- id: int
 - nombre: String
 - email: String
- }

class Pedido {

- fechaPedido: Date
- total: double
- estado: String
- direccionEnvio: String
- calcularTotal(): double

- confirmar(): void
- }

class LineaPedido {

- cantidad: int
- precioUnitario: double
- }

class Producto {

- codigo: String
- nombre: String
- precio: double
- stock: int
- decrementarStock(cantidad: int): void
- }

class Carrito {

- agregarProducto(p: Producto, cantidad: int): void
- vaciar(): void
- }

class Pago {

- fechaPago: Date
- importe: double
- metodo: String
- procesar(): boolean
- }

class ServicioCorreo {

- enviarConfirmacion(destinatario: String, pedido: Pedido): void
- }

class Catalogo {

- mostrarProductosDisponibles(): List
- }

' Relaciones

Cliente "1" -- "0.." Pedido : realiza

Pedido "1" -- "1.." LineaPedido : contiene

LineaPedido "1" -- "1" Producto : referencia

Pedido "1" -- "1" Pago : asociado a

Carrito "1" o-- "0.." Producto : agrega

Pedido ..> ServicioCorreo : usa para notificar

Catalogo ..> Producto : consulta

@enduml

???

Algunas anotaciones sobre este diagrama:

- He mantenido una clase **Catalogo** que no aparecía en la lista inicial de candidatos pero que surgió al asignar el método "mostrar catálogo" a un responsable. **Catalogo** actúa como un controlador que accede a los productos.
- **ServicioCorreo** no es una entidad del dominio sino un servicio técnico; su relación con **Pedido** es de dependencia porque **Pedido** probablemente lo invoca en el método **confirmar()** y no mantiene una referencia permanente.
- **Carrito** es un contenedor temporal con una relación de agregación hacia **Producto**, ya que los productos pueden existir sin el carrito.
- Las multiplicidades reflejan lo analizado: un cliente puede tener cero o más pedidos; un pedido tiene una o más líneas; cada línea referencia exactamente un producto; un pedido tiene exactamente un pago asociado.

4.7. Refinamiento con estereotipos

Dependiendo de la metodología que empleemos (por ejemplo, un enfoque de arquitectura en capas o el uso de estereotipos del Proceso Unificado), podemos añadir estereotipos como **<<entity>>**, **<<boundary>>** o **<<control>>** para clarificar el rol de cada clase. En PlantUML, los estereotipos se colocan entre **<<** y **>>** antes del nombre de la clase, o se definen con el comando **class Nombre <<Estereotipo>>**.

???

@startuml

class "Cliente" <>

class "Pedido" <>

class "Carrito" <>

class "Catalogo" <>

@enduml

???

Aunque no es obligatorio, ayuda a distinguir visualmente las clases que modelan el dominio de aquellas que gestionan la interfaz o la lógica de aplicación. Sin embargo, no debemos abusar; en muchos proyectos, un modelo de clases de dominio sin estereotipos es perfectamente suficiente.

4.8. Iteración y validación con los casos de uso

El modelo de clases no se termina en la primera pasada. Al revisar otros casos de uso del sistema, el modelo se enriquece. Por ejemplo, si existiese un caso de uso "Consultar Historial de Pedidos", añadiríamos probablemente un método **obtenerPedidosPorCliente(): List<Pedido>** en la clase **Cliente** o en un controlador. Si apareciera "Cancelar Pedido", añadiríamos un método **cancelar()** en **Pedido** y modificaríamos el atributo **estado**.

Cada nuevo caso de uso puede hacer crecer atributos y métodos, o incluso revelar la necesidad de nuevas clases. La clave es mantener la trazabilidad: si una clase o método no puede vincularse a ningún caso de uso, probablemente sea innecesaria.

Este ir y venir entre la vista funcional y la vista estructural es el latido del análisis y diseño orientado a objetos. PlantUML lo facilita porque ambos tipos de diagramas se escriben en archivos de texto plano que pueden versionarse y actualizarse sincronizadamente. Cuando un caso de uso cambia, el diagrama de clases puede modificarse en el mismo commit, manteniendo la coherencia de la documentación.

4.9. Errores frecuentes en la derivación

A lo largo de los años he visto algunos tropiezos recurrentes que conviene evitar:

- **Clases que deberían ser atributos:** a veces un sustantivo como "Dirección de envío" no necesita su propia clase si el sistema solo la almacena como un texto; podemos modelarla como un atributo de `Pedido`. Si la dirección tuviera estructura (calle, ciudad, código postal) y se reutilizase en varios lugares, entonces sí merecería ser una clase independiente.
- **Métodos huérfanos:** asignar un método a una clase que no tiene los datos para ejecutarlo. Por ejemplo, poner `enviarConfirmacion` en `Pedido` cuando los datos de conexión al servidor de correo están en `ServicioCorreo`. El método correcto es `Pedido.confirmar()`, que a su vez invoca a `ServicioCorreo.enviarConfirmacion()`.
- **Composición donde basta asociación:** recordar que la composición implica destrucción en cascada. Si no hay esa dependencia existencial, usen agregación o asociación simple.
- **Olvidar las restricciones de multiplicidad:** omitir la multiplicidad conduce a ambigüedades. Un desarrollador podría asumir que un pedido tiene un solo producto en lugar de muchos, o que un cliente solo puede tener un pedido. Las multiplicidades documentan decisiones de diseño importantes.

4.10. Cierre del proceso

Hemos recorrido el camino completo: desde la lectura atenta del caso de uso, pasando por la identificación de sustantivos, verbos y frases de pertenencia, asignándolos a clases, atributos, métodos y relaciones, y plasmándolos finalmente en un diagrama de clases de PlantUML. Este es, en esencia, el proceso que siguen los ingenieros de software cuando pasan del *qué* al *cómo*.

En los siguientes temas de este bloque, completaremos el modelo de clases con las nociones de interfaces, clases abstractas y organización en paquetes, y repasaremos las buenas prácticas de notación y mantenimiento. Pero el corazón del análisis estructural ya lo tienen: se trata de escuchar a los casos de uso y traducir sus historias en la arquitectura que las hará posibles.

5. Multiplicidad y navegabilidad: afinando las conexiones entre clases

Hemos construido el esqueleto de nuestro modelo estructural: tenemos clases, atributos, métodos y sabemos cómo relacionarlas mediante asociaciones, agregaciones, composiciones, herencias y dependencias. Pero ese esqueleto aún es tosco. Decir que un `Cliente` está asociado con un `Pedido` es un

avance, pero no responde a preguntas cruciales para el desarrollador: ¿un cliente puede tener muchos pedidos o solo uno? ¿un pedido pertenece obligatoriamente a un cliente? ¿desde un pedido puedo obtener directamente el cliente que lo realizó, o solo desde el cliente puedo llegar a sus pedidos? La **multiplicidad** y la **navegabilidad** son las herramientas que UML nos proporciona para refinar esas conexiones hasta convertirlas en instrucciones precisas de diseño.

En este tema desgranaremos ambos conceptos con la profundidad que merecen, veremos cómo expresarlos en PlantUML y cómo derivarlos del análisis funcional de los casos de uso. Una vez que dominen la multiplicidad y la navegabilidad, sus diagramas de clases dejarán de ser meros bocetos y se transformarán en auténticos planos de implementación.

5.1. Multiplicidad: cuántos objetos participan en la relación

La multiplicidad (también llamada cardinalidad) especifica el número de instancias de una clase que pueden estar vinculadas a una instancia de la otra clase en una asociación determinada. Se indica en cada extremo de la línea de asociación, junto a la clase correspondiente, y se lee en dirección contraria a la clase: si junto a **Pedido** aparece **1**, significa que un **Pedido** está asociado exactamente a un **Cliente**.

Los valores de multiplicidad que permite UML son:

- **1** : exactamente uno.
- **0..1** : cero o uno (opcional).
- **0..*** o simplemente ***** : cero o muchos.
- **1..*** : al menos uno, puede ser muchos.
- **n** : un número fijo (por ejemplo, **2** para los dos titulares de una cuenta mancomunada).
- **n..m** : un rango concreto (por ejemplo, **3..5**).

PlantUML acepta estas notaciones directamente, colocándolas entre comillas en el extremo correspondiente de la relación, como vimos en el tema anterior. Veamos algunos ejemplos aplicados a nuestro dominio de comercio electrónico.

???

@startuml

class Cliente

class Pedido

class Producto

class LineaPedido

class DireccionEnvio

Cliente "1" -- "0..*" Pedido : realiza

Pedido "1" -- "1.." LineaPedido : contiene

LineaPedido "1" -- "1" Producto : referencia

Cliente "1" -- "0..1" DireccionEnvio : tiene

@enduml

???

Interpretemos cada multiplicidad:

- **Cliente "1" -- "0..*" Pedido** : un pedido pertenece exactamente a un cliente; un cliente puede tener cero o más pedidos. La multiplicidad **0..*** junto a **Pedido** significa que un cliente recién registrado no tiene aún pedidos, y puede acumular muchos.
- **Pedido "1" *-- "1..*" LíneaPedido** : una línea de pedido pertenece exactamente a un pedido (composición); un pedido debe tener al menos una línea (porque un pedido vacío no tiene sentido) y puede tener muchas.
- **LíneaPedido "1" -- "1" Producto** : una línea de pedido referencia exactamente un producto; un producto puede ser referenciado por muchas líneas de pedido (multiplicidad ***** en el otro extremo se omite por claridad, pero implícitamente es ***** si no se indica).
- **Cliente "1" -- "0..1" DirecciónEnvío** : un cliente puede tener una dirección de envío por defecto, o ninguna (si aún no la ha proporcionado). La dirección pertenece a un cliente en concreto.

La multiplicidad no es un adorno: define restricciones que el sistema debe hacer cumplir. Si indicamos que un **Pedido** debe tener al menos una **LíneaPedido**, el código deberá impedir la creación de pedidos vacíos. Si un **Cliente** puede tener como máximo una **DirecciónEnvío**, la interfaz no debería permitir añadir una segunda sin antes eliminar la primera.

5.2. Cómo derivar la multiplicidad desde los casos de uso

Los casos de uso nos dan pistas valiosas sobre las multiplicidades. Frases como "el cliente puede consultar todos sus pedidos" sugieren que un cliente puede tener múltiples pedidos (multiplicidad **0..***). "Cada pedido incluye al menos un producto" sugiere **1..*** en el extremo de **LíneaPedido**. "Un cliente puede guardar una dirección de envío preferida" apunta a **0..1**.

Las postcondiciones también ayudan. Si al finalizar "Realizar Pedido" se indica que "el pedido queda registrado con un cliente asociado", confirmamos que la asociación **Cliente-Pedido** tiene multiplicidad **1** en el extremo del cliente.

No existe un algoritmo infalible, pero la combinación de sentido común, conocimiento del dominio y discusión en equipo permite establecer multiplicidades realistas. Y como todo en PlantUML, si más adelante se descubre que una multiplicidad era incorrecta, se modifica en el archivo **.puml** y el diagrama se actualiza al instante.

5.3. Navegabilidad: la dirección del conocimiento

Si la multiplicidad responde a "¿cuántos?", la navegabilidad responde a "¿quién conoce a quién?". En una asociación entre dos clases, podemos decidir que el vínculo sea recorrible en un solo sentido, en ambos, o en ninguno (asociación no navegable, rara en la práctica). La navegabilidad se representa con una punta de flecha en el extremo de la línea, apuntando hacia la clase que "es conocida".

En PlantUML, la navegabilidad se indica añadiendo **>** o **<** en la definición de la línea. Así, **ClaseA --> ClaseB** dibuja una flecha desde A hacia B, significando que desde A se puede acceder a B, pero no necesariamente al revés. Si se desea navegabilidad bidireccional, se omite la punta de flecha (solo **--**).

Veamos un ejemplo:

???

@startuml

```
class Pedido
class Cliente
class LineaPedido
class Producto
```

```
Pedido "1" --> "1" Cliente : pertenece a
Pedido "1" --> "1.." LineaPedido : contiene
LineaPedido "1" --> "1" Producto : referencia
@enduml
???
```

En este modelo, desde un **Pedido** puedo navegar hacia su **Cliente** (la flecha apunta a **Cliente**), pero no al revés: un **Cliente** no tiene una referencia directa a sus pedidos; si necesito obtener los pedidos de un cliente, tendré que buscarlos mediante una consulta. Esto es una decisión de diseño: desacoplamos **Cliente** de **Pedido** para que **Cliente** no acumule una colección potencialmente enorme.

Desde **Pedido** navego hacia **LineaPedido** (composición con navegabilidad unidireccional), y desde **LineaPedido** hacia **Producto**. De nuevo, un **Producto** no conoce todas las líneas de pedido que lo referencian; esa información se obtiene por otra vía si es necesario.

Si quisiéramos navegabilidad bidireccional entre **Pedido** y **Cliente**, omitiríamos la flecha:

```
???
@startuml
class Pedido
class Cliente
Pedido "1" -- "1" Cliente : pertenece a
@enduml
???
```

Esto implica que tanto **Pedido** conoce a su **Cliente** como **Cliente** conoce sus **Pedidos**. Ambas clases tendrán referencias mutuas en el código.

5.4. Derivación de la navegabilidad desde los casos de uso

El caso de uso nos permite responder a la pregunta: en un paso dado, ¿qué objeto necesita acceder a cuál? En "Realizar Pedido", cuando el sistema debe "mostrar el historial de pedidos de un cliente", necesitamos navegar desde **Cliente** a **Pedido**; eso sugiere navegabilidad de **Cliente** hacia **Pedido**. Pero si más adelante otro caso de uso requiere que, dado un pedido, se localice al cliente para enviarle una notificación, entonces necesitamos navegabilidad inversa. La decisión puede ser bidireccional para simplificar, o unidireccional si queremos minimizar acoplamiento.

En la práctica, muchos diseñadores optan por navegabilidad bidireccional en las asociaciones del dominio (por ejemplo, entre **Pedido** y **LineaPedido**, o entre **Cliente** y **Pedido**) porque la navegación suele ser necesaria en ambos sentidos. La navegabilidad unidireccional es más común en dependencias hacia servicios técnicos o en relaciones de uso puntual, donde el objeto usado no necesita conocer a quien lo usa.

5.5. Combinando multiplicidad y navegabilidad

Ambos conceptos se combinan en la misma notación. La multiplicidad se coloca junto al extremo de la clase, la navegabilidad se deduce de la punta de flecha. Un extremo puede tener flecha o no, y llevar una multiplicidad. PlantUML permite expresar todo junto con claridad:

```
???
@startuml
class Departamento
class Empleado
Departamento "1" o--> "5..*" Empleado : agrega
@enduml
???
```

Aquí, un **Departamento** agrega de cinco a muchos **Empleados** (agregación), y desde **Departamento** se puede navegar hacia sus empleados (flecha), pero no al revés. Un empleado no conoce directamente su departamento (quizás se obtiene mediante un repositorio).

Otro ejemplo con navegabilidad bidireccional:

```
???
@startuml
class CuentaBancaria
class Titular
CuentaBancaria "1" -- "1..2" Titular : pertenece
@enduml
???
```

Una cuenta bancaria puede tener uno o dos titulares, y la asociación es navegable en ambos sentidos: desde la cuenta puedo acceder a los titulares, y desde un titular puedo acceder a sus cuentas.

5.6. Buenas prácticas en multiplicidad y navegabilidad

- **No sobrecargar de flechas:** si todas las asociaciones son bidireccionales, el diagrama puede volverse confuso. Reserven las flechas para indicar restricciones de diseño conscientes.
- **La multiplicidad debe reflejar las reglas de negocio:** antes de escribir **0..*** o **1..***, pregúntense qué permite el negocio. Un pedido sin líneas quizás sea válido como borrador; entonces **0..*** en **Pedido-LineaPedido** sería correcto. Si no, debe ser **1..***.
- **Revisar las multiplicidades con los stakeholders:** un analista de negocio puede confirmar si realmente un cliente puede tener un número ilimitado de pedidos o si hay un límite.
- **Documentar las decisiones:** si optan por navegabilidad unidireccional para reducir acoplamiento, anótenlo en una nota en el diagrama o en la documentación de diseño. Otros desarrolladores lo agradecerán.
- **Actualizar el modelo con cada nuevo caso de uso:** un nuevo caso de uso puede requerir navegabilidad inversa o modificar una multiplicidad. El diagrama de clases debe evolucionar con el proyecto.

5.7. Representación en PlantUML: detalles avanzados

La guía de PlantUML ofrece variantes para personalizar la visualización de las relaciones (páginas 83-86), incluyendo líneas de estilo `[bold]`, `[dashed]`, `[dotted]`, colores y grosores. Esto puede usarse para resaltar asociaciones con multiplicidades restrictivas o navegabilidades importantes. Por ejemplo:

```
???  
@startuml  
class Pedido  
class Cliente  
Pedido "1" -[bold]-> "1" Cliente : pertenece  
@enduml  
???
```

Aunque no es necesario para la comprensión del modelo, en presentaciones ejecutivas puede ayudar a dirigir la atención hacia las relaciones críticas.

5.8. Más allá del diagrama: impacto en la implementación

La multiplicidad y la navegabilidad tienen consecuencias directas en el código. Si entre `Pedido` y `LineaPedido` definimos composición con multiplicidad `1..*` y navegabilidad unidireccional desde `Pedido`, el código resultante tendrá en `Pedido` una colección de `LineaPedido` que se inicializa en el constructor, y posiblemente métodos `agregarLinea` y `eliminarLinea`. No existirá una referencia inversa desde `LineaPedido` a `Pedido`. Si más adelante se necesita, se puede añadir sin romper el modelo, pero es más costoso.

Por eso es importante tomar estas decisiones con criterio durante el diseño. PlantUML, al ser código, nos permite simular estos escenarios antes de implementarlos: si vemos que un diagrama resulta incómodo porque hay que añadir muchas flechas bidireccionales, quizás sea una señal de que el diseño está demasiado acoplado y conviene repensarlo.

Con esto completamos el afinamiento de las conexiones entre clases. En el próximo tema abordaremos las clases abstractas, las interfaces y la organización en paquetes, que nos permitirán elevar nuestro modelo estructural al nivel de abstracción que los sistemas complejos exigen.

6. Clases abstractas, interfaces y paquetes: elevando la abstracción del modelo estructural

Hasta ahora, nuestro modelo de clases se ha construido con clases concretas, atributos y métodos perfectamente definidos, y relaciones que reflejan el tejido del dominio. Pero a medida que el sistema crece, aparecen patrones que exigen un nivel superior de abstracción: comportamientos comunes que queremos garantizar sin obligar a una implementación única, o agrupaciones lógicas que eviten que el diagrama se convierta en un catálogo interminable. UML, y por tanto PlantUML, nos proporcionan tres mecanismos para manejar esta complejidad: las **clases abstractas**, las **interfaces** y los **paquetes**.

En este tema, exploraremos cada uno de ellos con la profundidad que merecen, veremos cómo se declaran y se visualizan en PlantUML según la guía de referencia (páginas 66-68 para clases abstractas e interfaces; páginas 72-75 para paquetes), y comprenderemos por qué son herramientas indispensables para cualquier ingeniero de software que aspire a diseñar sistemas mantenibles y extensibles.

6.1. Clases abstractas: el molde que no se instancia

Una clase abstracta es una clase que **no puede ser instanciada directamente**. Su propósito es servir como plantilla para otras clases (sus subclases), proporcionándoles una estructura común de atributos y métodos, algunos de los cuales pueden estar implementados y otros no. Los métodos no implementados —métodos abstractos— deben ser definidos obligatoriamente por cada subclase concreta, garantizando así un comportamiento polimórfico.

En UML, una clase abstracta se distingue visualmente porque su nombre y, a menudo, sus métodos abstractos, aparecen en **cursiva**. PlantUML sigue esta convención y, además, permite marcarlas explícitamente con los modificadores `{abstract}` o con la palabra reservada `abstract class`.

¿Cuándo usar una clase abstracta? Cuando tenemos un conjunto de clases que comparten una parte significativa de implementación, pero también requieren que cada una personalice ciertos comportamientos. Por ejemplo, en un sistema de notificaciones, podríamos tener una clase abstracta `Notificador` con un método concreto `enviarResumenDiario()` y un método abstracto `formatearMensaje()`. Las subclases `NotificadorEmail` y `NotificadorSMS` heredarían el primer método y proporcionarían su propia implementación del segundo.

Veamos cómo se expresa esto en PlantUML:

```
???\n@startuml\nabstract class Notificador {\n\n    • destinatario: String\n\n    • enviarResumenDiario(): void\n      {abstract} + formatearMensaje(): String\n    }\n\nclass NotificadorEmail extends Notificador {\n\n    • formatearMensaje(): String\n    }\n\nclass NotificadorSMS extends Notificador {\n\n    • formatearMensaje(): String\n    }\n@enduml\n???
```

En este ejemplo, `Notificador` es abstracta (su nombre está en cursiva) y declara un método abstracto `formatearMensaje()`. Las subclases `NotificadorEmail` y `NotificadorSMS` heredan el atributo `destinatario` y el método `enviarResumenDiario()` ya implementado, pero deben proporcionar su propia versión de `formatearMensaje()`. Cualquier intento de instanciar `Notificador` directamente será rechazado por el compilador.

La guía de PlantUML (página 66) también permite usar la palabra reservada **abstract** antes del nombre de la clase, sin necesidad de las llaves **{abstract}** en cada método, lo cual es más compacto:

```
???  
@startuml  
abstract class Notificador  
class NotificadorEmail extends Notificador  
class NotificadorSMS extends Notificador  
@enduml  
???
```

Ambas notaciones son válidas y producen el estilo visual adecuado.

6.2. Interfaces: el contrato puro

Si una clase abstracta es un molde que puede contener implementación, una interfaz es un **contrato puro**: no proporciona ninguna implementación, solo declara un conjunto de métodos que las clases que la realicen deben implementar obligatoriamente. En UML, una interfaz se puede representar de dos maneras: como una clase estereotipada **<<interface>>** (un rectángulo con el estereotipo) o como un pequeño círculo (notación "lollipop") conectado a la clase que la implementa.

PlantUML soporta ambas representaciones (páginas 67 y 76). La declaración con **interface** genera el rectángulo estereotipado; la notación de círculo se consigue usando el símbolo **()** para declarar la interfaz y luego conectándola a la clase con la relación de realización (**<|..**).

¿Cuándo usar una interfaz en lugar de una clase abstracta? La regla de oro es: si lo que queremos es compartir implementación, usamos clase abstracta. Si lo que queremos es definir un comportamiento que múltiples clases no relacionadas jerárquicamente deben cumplir, usamos interfaz. Una misma clase puede implementar múltiples interfaces, pero solo puede heredar de una clase abstracta. Esta flexibilidad hace de las interfaces la herramienta preferida para definir contratos transversales.

Un ejemplo clásico es la interfaz **IPagable** en un sistema de facturación:

```
???  
@startuml  
interface IPagable {  
    • calcularImporte(): double  
    • procesarPago(): boolean  
}  
  
class Factura implements IPagable  
class Recibo implements IPagable  
class NotaCredito implements IPagable  
@enduml  
???
```

Factura, **Recibo** y **NotaCredito** son clases muy diferentes entre sí, posiblemente con distintas jerarquías de herencia, pero todas comparten la obligación de saber calcular su importe y procesar su pago. La

interfaz **IPagable** les exige cumplir ese contrato sin imponerles ninguna estructura interna.

La notación de "lollipop" es especialmente útil cuando queremos mostrar la interfaz de forma compacta, sin ocupar un rectángulo completo. En PlantUML, se consigue definiendo la interfaz con **()** y conectándola con la relación de realización:

```
???\n@startuml\n() IPagable\nclass Factura\nclass Recibo\nFactura ..|> IPagable\nRecibo ..|> IPagable\n@enduml\n???
```

Esta vista es frecuente en diagramas de componentes o de despliegue, pero también puede emplearse en diagramas de clases muy poblados para ahorrar espacio.

6.3. Clases abstractas versus interfaces: una decisión de diseño

Como ingenieros de software, deben saber cuándo elegir una u otra. Comparto una tabla que utilizo en mis sesiones de diseño con el equipo:

Criterio	Clase abstracta	Interfaz
Proporciona implementación	Sí (parcial o total)	No (solo firmas)
Herencia múltiple	No (una sola clase padre)	Sí (múltiples interfaces)
Constructor	Sí	No
Atributos	Sí	No (solo constantes estáticas en algunos lenguajes)
Relación semántica	"es-un" (herencia)	"se comporta como" (contrato)
Evolución	Añadir método con implementación no rompe subclases	Añadir método rompe todas las clases que la implementan (en lenguajes sin métodos por defecto)

En la práctica, muchos diseños modernos tienden a preferir interfaces combinadas con composición (principio de "composición sobre herencia"), reservando las clases abstractas para situaciones donde realmente haya una implementación común que merezca ser reutilizada.

6.4. Paquetes: organizando el modelo de clases

Si las clases abstractas y las interfaces gestionan la complejidad dentro del modelo, los paquetes la gestionan desde fuera, agrupando clases relacionadas en módulos con significado propio. Un paquete en

UML es un mecanismo de propósito general para organizar elementos, y en el diagrama de clases funciona como un espacio de nombres que contiene clases, interfaces, enumeraciones y otros paquetes.

Los paquetes son esenciales cuando el sistema alcanza unas pocas decenas de clases. Sin ellos, el diagrama se convierte en una sopa de rectángulos donde es imposible orientarse. Con paquetes, podemos dividir el modelo en subsistemas lógicos: "Dominio", "Persistencia", "Servicios", "Interfaz de Usuario", etc.

En PlantUML, los paquetes se declaran con la palabra reservada `package`, y su contenido se delimita con llaves `{}`. Podemos anidar paquetes y también definir el estilo visual (`rectangle`, `folder`, `node`, `cloud`, `frame`, `database`) mediante `skinparam packageStyle` o mediante estereotipos individuales (páginas 72-75).

Veamos un ejemplo que organiza nuestro modelo de comercio electrónico:

```
???  
@startuml  
package "Dominio" {  
class Cliente  
class Pedido  
class LineaPedido  
class Producto  
class Pago  
}  
  
package "Servicios" {  
class ServicioCorreo  
class ServicioPago  
}  
  
package "Interfaz" {  
class Catalogo  
class Carrito  
}  
  
Cliente --> Pedido  
Pedido *--> LineaPedido  
LineaPedido --> Producto  
Pedido ..> ServicioCorreo  
Pedido ..> ServicioPago  
Catalogo ..> Producto  
Carrito o--> Producto  
@enduml  
???
```

Aquí, el modelo se ha dividido en tres paquetes que reflejan una arquitectura en capas simplificada. Las clases de dominio contienen la lógica de negocio y las entidades; los servicios encapsulan integraciones

externas; la interfaz agrupa los controladores que median entre el usuario y el dominio. Esta organización no solo clarifica el diagrama, sino que también prefigura la estructura de directorios del código fuente.

Los paquetes pueden relacionarse entre sí mediante dependencias (`..>`), indicando que un paquete necesita elementos de otro. Por ejemplo:

```
???  
@startuml  
package "Interfaz" {  
class Catalogo  
}  
package "Dominio" {  
class Producto  
}  
package "Servicios" {  
class ServicioPago  
}  
  
Interfaz ..> Dominio : usa  
Interfaz ..> Servicios : usa  
@enduml  
???
```

Esta vista arquitectónica de alto nivel comunica las dependencias entre subsistemas sin entrar en el detalle de cada clase. Es un magnífico punto de partida para discutir la estructura del código con el equipo.

6.5. Cómo se derivan estos elementos desde los casos de uso

Llegados a este punto, podemos cerrar el círculo con los casos de uso. Las clases abstractas y las interfaces no suelen aparecer directamente en la especificación textual, sino que surgen durante el diseño como abstracciones que unifican conceptos dispersos. Si en varios casos de uso aparece la necesidad de pagar de diferentes formas, deducimos una interfaz `MetodoPago`. Si varias clases comparten atributos y métodos idénticos, extraemos una clase abstracta.

Los paquetes, por su parte, se corresponden a menudo con las agrupaciones funcionales que ya identificamos en los diagramas de casos de uso (por ejemplo, un paquete "Ventas" que contiene los casos de uso "Realizar Pedido", "Consultar Historial" y "Cancelar Pedido" probablemente albergará las clases de dominio que implementan esas funcionalidades).

De nuevo, la trazabilidad es la clave: cada clase, interfaz o paquete del diagrama estructural debería poder vincularse a uno o varios casos de uso que justifiquen su existencia.

Dominar las clases abstractas, las interfaces y los paquetes completa su capacidad para modelar la estructura de cualquier sistema. En el último tema de este bloque, recopilaremos las buenas prácticas de notación, mantenimiento y evolución que garantizan que sus diagramas de clases sigan siendo un activo valioso a lo largo de todo el ciclo de vida del software.

7. Buenas prácticas y cierre: el arte de mantener vivo el modelo estructural

Hemos recorrido un camino exhaustivo por el modelado estructural con UML y PlantUML. Desde la definición de clases, atributos y métodos, pasando por la riqueza de las relaciones, la derivación metódica desde los casos de uso, hasta el afinamiento con multiplicidades, navegabilidades, clases abstractas, interfaces y paquetes. Ahora, antes de poner punto final a este bloque, es imprescindible consolidar una serie de buenas prácticas que aseguren que el diagrama de clases no se convierta en un adorno que se abandona tras la primera iteración, sino en un artefacto vivo, útil y respetado por todo el equipo.

A lo largo de mi carrera, he visto modelos de clases impecables en su concepción inicial degenerar en mentiras gráficas porque nadie los actualizaba, o peor aún, modelos tan recargados de detalles irrelevantes que los desarrolladores preferían ignorarlos. La diferencia entre un diagrama que suma y uno que estorba radica en la disciplina con que se aplican ciertas prácticas de notación, organización y mantenimiento. En este tema de cierre, quiero compartir esas prácticas con ustedes, junto con una reflexión final sobre el valor duradero de esta herramienta.

7.1. Mantener el foco en el objetivo del modelo

Antes de dibujar una sola línea, debemos preguntarnos: ¿para quién es este diagrama y qué decisión debe informar? No es lo mismo un diagrama de clases de **análisis** —orientado a comunicar el dominio del problema a los expertos de negocio— que un diagrama de clases de **diseño** —orientado a guiar la implementación por parte de los desarrolladores—. Y ambos son distintos de un diagrama de clases de **arquitectura**, que muestra solo los paquetes principales y sus dependencias.

Cada nivel tiene su propio nivel de detalle. Un diagrama de análisis puede omitir tipos de datos precisos, visibilidades y métodos de infraestructura. Un diagrama de diseño, en cambio, se beneficiará de incluir esos detalles. La primera buena práctica es, por tanto, **elegir deliberadamente el nivel de abstracción y mantenerlo consistente** en todo el diagrama. Mezclar clases de análisis con detalles de implementación confunde al lector y desdibuja el propósito del modelo.

7.2. Notación clara y consistente

La notación UML es un lenguaje, y como todo lenguaje, tiene dialectos y variantes. Para que el equipo se comunique sin ruido, es crucial establecer convenciones y respetarlas. Algunas que yo impongo desde el inicio del proyecto son:

- **Nombres de clases:** singular, en CamelCase (p. ej., `LineaPedido`). Evitar nombres ambiguos o excesivamente genéricos (`Gestor`, `Procesador`, `Util`).
- **Nombres de atributos y métodos:** en camelCase, comenzando con minúscula (p. ej., `calcularTotal()`). Los métodos booleanos pueden llevar prefijos como `es`, `tiene` (p. ej., `esVip(): boolean`).
- **Visibilidad:** decidir si se mostrarán los iconos de visibilidad (+, -, #, ~) o si se ocultarán mediante `skinparam classAttributeIconSize 0`. Si se muestran, deben ser correctos y coherentes con la implementación prevista.

- **Multiplicidades:** colocarlas siempre en ambos extremos de la asociación, incluso si uno de ellos es 1. Una multiplicidad omitida es una ambigüedad.
- **Uso de estereotipos:** limitado a los que realmente aportan información (`<<entity>>`, `<<control>>`, `<<boundary>>`, `<<repository>>`). No inventar estereotipos sin consenso en el equipo.
- **Colores:** usarlos con moderación. A menudo, un fondo suave para los paquetes y colores estándar para las clases bastan. La guía de PlantUML (páginas 79-81) muestra cómo aplicar colores mediante `skinparam` o directamente en la definición de la clase (`#color`), pero recomendando no abusar.

7.3. Derivar, no inventar: la trazabilidad con los casos de uso

Ya insistí en esto en el tema 4, pero es tan importante que merece su lugar en las buenas prácticas. Todo elemento del diagrama de clases —clase, atributo, método, relación— debe poder rastrearse hasta uno o varios casos de uso. Esta trazabilidad:

- Justifica la existencia de cada clase frente a los stakeholders.
- Facilita la estimación de impacto cuando un caso de uso cambia.
- Evita la proliferación de clases "por si acaso" que nunca se implementan.
- Permite validar el modelo preguntando: "si elimino esta clase, ¿qué caso de uso deja de funcionar?".

En PlantUML, una ayuda sencilla es incluir en una nota el código del caso de uso del que procede una clase, o mantener una tabla de trazabilidad externa que vincule artefactos. Pero lo fundamental es que mentalmente, como diseñadores, mantengamos ese cordón umbilical entre funcionalidad y estructura.

7.4. La granularidad adecuada: ni demasiado, ni demasiado poco

Un diagrama de clases debe contener las clases necesarias para transmitir la idea central, y nada más. Caer en la "parálisis por análisis" intentando modelar cada clase del sistema con cada atributo y cada método es contraproducente. El diagrama se vuelve ilegible y nadie lo mantiene.

Como regla general, un diagrama de clases debería caber en una pantalla o en una página impresa. Si necesitamos más de 20 o 30 clases para contar la historia, probablemente estamos abarcando demasiado. En ese caso, usamos **paquetes** para crear vistas de más alto nivel, y luego elaboramos diagramas de clases detallados para cada paquete por separado. PlantUML facilita esto porque podemos tener múltiples archivos `.puml` que incluyan partes del modelo mediante `!include`.

7.5. Organización en paquetes significativa

La organización en paquetes no debe ser arbitraria ni basarse únicamente en criterios técnicos. Como vimos en el tema 6, los paquetes deben reflejar la cohesión lógica del dominio o, en su defecto, una arquitectura en capas bien definida. Algunos principios que aplico son:

- **Paquete por módulo funcional:** "Ventas", "Compras", "Almacén", "Facturación". Esto suele alinearse con los paquetes de casos de uso.
- **Paquete por capa técnica:** "Interfaz", "Aplicación", "Dominio", "Infraestructura". Esta organización es más cercana al diseño y a la implementación, y ayuda a visualizar la separación de responsabilidades.

- **Evitar paquetes "Miscelánea" o "Utilidades":** son un imán para clases sin hogar y un síntoma de que la taxonomía necesita revisión.

Cualquiera que sea el criterio, lo importante es que todo el equipo lo entienda y lo comparta. En la documentación del proyecto, podemos incluir una breve leyenda o un comentario en el archivo `.puml` que explique la convención.

7.6. Autodisciplina en el uso de relaciones

Las relaciones son la salsa del modelo, pero también su perdición. He visto diagramas en los que cada clase está conectada con todas las demás mediante una maraña de asociaciones, dependencias y herencias. Algunas pautas para mantener la limpieza:

- **Asociación solo si hay vínculo estructural permanente.** Si una clase usa a otra solo temporalmente, mejor una dependencia (`..>`).
- **Composición solo si la parte no puede vivir sin el todo.** Si hay duda, usar agregación (`o--`) o asociación simple (`--`).
- **Herencia solo si existe relación "es-un" genuina.** No forzarla para reutilizar código si no hay una jerarquía conceptual clara.
- **Interfaces para contratos, no para simular herencia múltiple sin criterio.**
- **No duplicar relaciones:** si una asociación ya implica una dependencia, no dibujar ambas. El diagrama debe ser minimalista.

7.7. Mantenimiento continuo y versionado

El diagrama de clases es un artefacto de software más, y como tal, debe someterse a control de versiones y actualizarse con cada cambio significativo. Mis recomendaciones son:

- **Almacenar los archivos `.puml` en el mismo repositorio que el código fuente.** Así, cuando se modifica una clase, el diagrama puede actualizarse en el mismo commit.
- **Revisar el diagrama en las reuniones de diseño:** cuando se discute un nuevo caso de uso o un cambio en uno existente, actualizar el diagrama de clases en tiempo real proyectando PlantUML. Esto fomenta la participación y mantiene el modelo sincronizado con las decisiones.
- **No aspirar a la perfección inmediata:** el modelo puede comenzar con clases sin atributos ni métodos, e ir refinándose iteración a iteración. Es preferible un diagrama incompleto pero actualizado que uno completísimo pero obsoleto.
- **Usar `!include` para modularizar:** dividir el modelo en varios archivos (por paquete, por capa, etc.) y combinarlos mediante la directiva `!include`. Esto facilita la edición y la reutilización.

7.8. Aprovechar las capacidades de PlantUML para la legibilidad

La guía de PlantUML nos brinda herramientas que, bien empleadas, mejoran sustancialmente la presentación:

- **Separadores y títulos dentro de clases (`..`, `==`, `--`, `__`):** usar para agrupar atributos y métodos lógicamente, en lugar de presentar una lista plana interminable.
- **Notas y comentarios:** tanto en el código fuente (con `'`) como en el diagrama (con `note`), para explicar decisiones de diseño, restricciones o enlaces a la especificación de casos de uso.

- **Estilos globales:** definir un estilo base con `skinparam` y aplicarlo consistentemente en todos los diagramas del proyecto. Esto comunica profesionalidad y coherencia.
- **Dirección del diagrama:** `left to right direction` o `top to bottom direction` según convenga. Probar ambas para ver cuál aprovecha mejor el espacio.

7.9. Errores frecuentes al mantener diagramas de clases

Recojo aquí, a modo de lista de verificación, los errores más habituales que he tenido que corregir en revisiones de diseño:

- **Diagrama desconectado de la realidad:** las clases no coinciden con las que realmente existen en el código. El diagrama se ha convertido en un documento histórico.
- **Sobrecarga de detalles de implementación:** incluir clases de infraestructura, DTOs, mapeadores, configuración... elementos que pertenecen a otros artefactos o que varían demasiado rápido.
- **Asociaciones sin nombres ni multiplicidad:** dejan al lector adivinando la naturaleza de la relación.
- **Uso incorrecto de la composición:** tratar como composición cualquier relación todo-parte. Recordar la pregunta: ¿la parte sobrevive sin el todo?
- **Atributos modelados como clases:** por ejemplo, crear una clase `Nombre` con un solo atributo `String`. Si el concepto no tiene comportamiento ni vida propia, probablemente sea un atributo.
- **Circularidad en las dependencias entre paquetes:** si el paquete A depende de B y B depende de A, hay un acoplamiento cíclico que suele ser síntoma de una mala descomposición.

7.10. Reflexión final: el diagrama de clases como lenguaje común

Llegamos al final de este bloque sobre modelado estructural. Hemos construido, paso a paso, la capacidad de plasmar la anatomía de un sistema software en un diagrama preciso, mantenible y comunicativo. Pero no quiero que se vayan con la idea de que el diagrama de clases es un fin en sí mismo. Es una herramienta, un lenguaje que permite a los ingenieros de software debatir, acordar y documentar la arquitectura interna de un sistema antes de escribir código, y también después, para recordar por qué se tomaron ciertas decisiones.

Cuando modelan con PlantUML, están haciendo algo más que dibujar: están escribiendo un texto que es simultáneamente documentación y fuente de generación de imágenes. Están aplicando prácticas de ingeniería (control de versiones, modularidad, revisión por pares) al diseño del software. Están, en definitiva, tratando el diseño como código.

Les animo a que practiquen. Tomen los casos de uso que modelaron en el bloque anterior. Deriven sus clases. Discutan con sus compañeros si una relación es composición o agregación. Rompan el diagrama, modifíquelo, compárenlo con el código que escriben en sus proyectos. Solo así, con la práctica deliberada y la reflexión constante, se convertirán en los arquitectos de software que la industria necesita.

El diagrama de clases es suyo ahora. Úsenlo con criterio, manténganlo vivo y verán cómo se transforma en uno de los aliados más poderosos para construir sistemas robustos y bien comunicados.